

基于视觉-文本双模态融合的直播场景情感识别系统实验报告

实验者：王文淼2024141450140 任晨曦2024141450142 王浩然2024141450146 李奕奋2024141450123 张哲睿2024141450113

2026 年 6 月 21 日

摘要

本实验搭建并实现视觉-文本双模态融合的直播场景情感识别系统，面向直播主播表情与弹幕、字幕开展多模态情感分析。视觉分支采用 YuNet 轻量化人脸检测器定位主播面部区域，基于 ResNet18 构建 EmotionResNet 模型完成七类基础表情识别（开心、悲伤、愤怒、恐惧、厌恶、惊讶、中性）；文本分支通过 EasyOCR 提取视频内字幕、弹幕文本，结合自研 EnhancedOfflineBERT 模型实现正向、负向、中性三分类情感识别。融合模块设计基于单分支置信度的动态加权晚期融合策略，将文本三分类结果映射至七维情绪空间，与视觉情绪概率向量加权融合输出最终情感。实验针对单模态缺陷、多模态互补性、表情与文本情感矛盾样本开展专项测试与分析。系统前端基于 Gradio 搭建轻量级 Web 交互界面，支持视频上传、实时推理流式显示与结果可视化。结果表明：多模态融合有效弥补单一模态受环境、语义干扰的短板，系统鲁棒性与识别准确率显著提升，可有效适配直播复杂场景下的情感识别需求。

1 实验目的

- 完成视觉单模态情感识别：实现视频流中主播人脸实时检测、面部表情七分类，分析光照、人脸姿态、遮挡对视觉识别效果的影响。
- 完成文本单模态情感识别：通过 OCR 提取直播字幕、弹幕文本，实现文本情感三分类，分析网络用语、短句、口语化表达对文本识别的干扰。
- 实现多模态晚期融合算法：设计动态加权融合规则，完成文本、视觉模态特征融合，输出统一情感结果。
- 对比分析单模态与多模态模型性能差异，验证多模态信息互补的优势。
- 针对表情-文本情感矛盾样本进行专项分析，测试动态融合策略在冲突场景下的决策能力。
- 基于完整工程代码实现系统落地，完成视频流实时推理、可视化展示等功能，验证系统工程实用性。

2 实验环境与软硬件配置

2.1 硬件环境

- 运行设备：PC 台式机

- 处理器：主流 x86 架构 CPU
- 显卡：支持 CUDA 的 NVIDIA 显卡（可选，代码自动兼容 CPU/GPU 双模式推理）
- 存储：本地路径存储模型权重、YuNet 人脸检测 ONNX 模型、测试视频文件

2.2 软件环境

- 操作系统：Windows 10/11
- 编程语言：Python 3.8+
- 核心依赖库：
 - 计算机视觉：OpenCV-python（人脸检测、视频解码、画面绘制）、PIL（中文文字渲染）
 - 深度学习框架：PyTorch（模型训练、推理、张量计算）、torchvision（ResNet 网络、图像预处理）
 - 文本识别：EasyOCR（文字提取，本地离线运行）
 - 前端交互：Gradio（Web 界面搭建、视频流式输出）
 - 辅助工具：re（文本清洗）、numpy（数值计算）

2.3 文件路径配置（代码对应）

所有模型、视频、OCR 模型均部署于本地 E:\python\project1 目录，全程离线运行，无网络依赖：

- YuNet 人脸检测模型：E:\python\project1\face_detection_yunet_2023mar.onnx
- 文本情感模型权重：E:\python\project1\best_danmu_model_v1.pth
- 视觉表情模型权重：E:\python\project1\best_emotion_model.pth
- EasyOCR 本地模型：E:\python\project1（关闭自动下载，纯离线识别）

3 系统总体架构与工作流程

3.1 整体架构

系统采用流水线式模块化设计，共分为 7 大功能模块（含前端交互），模块间低耦合、独立运行：

前端界面（Gradio）→ 视频输入模块 → 分支 1（人脸检测 + 视觉情感识别）、分支 2（OCR 文本提取 + 文本情感识别）→ 多模态融合决策模块 → 结果可视化与输出模块 → 前端实时画面刷新
核心功能模块拆解（结合代码类结构）：

- 前端交互模块（app.py）：基于 Gradio 构建 Web 界面，提供视频上传控件、启动按钮和实时画面显示区域，通过生成器函数与后端视频处理流水线对接。
- 视频解码模块：基于 cv2.VideoCapture 读取本地视频流，逐帧输出图像数据。

- **OCR 模块 (SyncOCR)**: 同步提取字幕、弹幕文本, 做文本清洗后输出纯文本内容。
- **人脸检测模块 (YuNetFaceDetector)**: 基于 YuNet ONNX 模型检测视频指定 ROI 区域内人脸, 无 YuNet 时自动降级为 Haar 级联检测。
- **视觉情感识别模块 (EmotionResNet)**: 对截取人脸灰度图做预处理, 完成七类表情概率预测。
- **文本情感识别模块 (EnhancedOfflineBERT + EnhancedTokenizer)**: 字符级分词、编码, 完成文本三分类情感预测。
- **多模态融合模块 (MultiModalEmotionFusion)**: 模态维度映射、动态加权融合、最终情感决策。
- **可视化与结果渲染模块**: 绘制人脸框、识别区域、中文结果, 将标注后的帧返回前端展示。

3.2 核心工作流程

1. 用户通过 Gradio 前端界面上传视频文件, 点击“开始实时分析”按钮。
2. 后端接收视频路径, 初始化所有模型和 OCR 工具, 进入逐帧处理循环。
3. 视频逐帧读取, 图像同时送入人脸检测与 OCR 处理。
4. OCR 模块处理画面, 分底部字幕区域、右侧弹幕区域提取文本并清洗; 主线程定时检测人脸区域, 截取人脸 ROI 图像。
5. 文本、视觉数据分别送入对应模型, 输出情感概率与单分支置信度。
6. 融合模块将文本三分类结果映射为七维情绪向量, 结合双方置信度动态计算权重, 加权融合得到最终情感标签。
7. 将人脸框、识别区域、原始文本、单模态结果、融合结果绘制在视频画面中。
8. 标注后的帧通过生成器逐次返回给前端, 前端实时更新显示画面, 形成流式播放效果。
9. 循环执行直至视频播放结束或用户手动停止。

3.3 ROI 区域划分 (代码固定区域策略)

为提升识别效率、减少无效计算, 对视频画面做固定比例区域划分:

- **字幕区域**: 画面纵向 70%至100%, 横向 0 至 100% (视频底部区域)
- **弹幕区域**: 画面横向 70% 至 100%, 纵向 0 至 70% (视频右侧区域)
- **主播人脸区域**: 画面全屏整块区域

4 核心任务一: 视觉情感识别 (主播表情识别)

4.1 技术方案选型

视觉任务分为人脸检测和表情分类两个子任务:

4.1.1 人脸检测：YuNet

选用 OpenCV 官方开源 YuNet 轻量化人脸检测模型（ONNX 格式），优势为体积小、速度快、适配视频实时流。代码中设置：置信度阈值 0.6、NMS 阈值 0.3，仅保留高置信度人脸。

容错机制：若 YuNet 模型缺失/加载失败，自动降级为 OpenCV 内置 Haar 级联人脸检测器，保证系统不中断。

优化策略：加入帧缓存降频机制，连续帧不重复检测人脸，降低 CPU 算力消耗。

4.1.2 表情分类：EmotionResNet（基于 ResNet18）

以经典轻量卷积网络 ResNet18 为骨干网络，针对表情任务做定制修改：

- **输入适配：**原 ResNet18 接收 RGB 三通道图像，本任务改为单通道灰度图，修改首层卷积 conv1 输入通道为 1
- **输出适配：**将原 ImageNet 千分类全连接层，替换为 7 维输出全连接层，对应 7 类表情
- **后处理：**模型输出 Logits 经 Softmax 归一化为概率向量，取最大概率为预测表情，最大值为该分支视觉置信度

4.2 图像预处理流程（代码对应 transforms）

截取人脸区域图像后执行标准化预处理：

- **缩放：**统一尺寸至 48×48
- **灰度化：**转为单通道图像
- **张量转换：**PIL 图像转为 PyTorch 张量
- **归一化：**均值 0.5、标准差 0.5，完成数据标准化

4.3 识别类别

七类表情标签：Angry（愤怒）、Disgust（厌恶）、Fear（恐惧）、Happy（开心）、Sad（悲伤）、Surprise（惊讶）、Neutral（中性）。

4.4 单模态视觉缺陷分析（结合实验现象 + 代码逻辑）

- **环境干扰：**直播画面光照不均、反光、逆光会大幅降低人脸特征质量，导致置信度下降
- **姿态与遮挡：**主播侧脸、低头、手部遮挡面部时，人脸检测漏检/误检，表情识别失效
- **帧率限制：**高频切换的表情易被帧缓存机制跳过，造成识别滞后

5 核心任务二：文本情感识别（字幕 + 弹幕识别）

5.1 技术方案选型

文本任务分为文本提取、分词编码、情感分类三部分，全程离线运行，无网络请求。

5.1.1 文本提取：EasyOCR

采用 EasyOCR 实现离线文字识别。

关键优化（代码实现）：

- 区域裁剪：仅对预设字幕、弹幕 ROI 做识别，减少计算量
- 图像增强：小尺寸文字区域做放大插值，提升小字识别率
- 文本清洗：通过正则表达式过滤标点符号、乱码，输出纯净文本
- 阈值过滤：仅保留置信度 > 0.15 的识别结果，过滤无效字符

5.1.2 文本编码：EnhancedTokenizer

自研字符级分词器，以字符为最小单元构建词表（适配中文无分词特点）：

- 内置占位符：<PAD>填充符、<UNK>未知字符
- 词表构建：基于训练语料统计字符频次，按频次排序生成词表
- 固定长度编码：统一文本长度为 32 字符，超长截断、短文本补零

5.1.3 情感分类：EnhancedOfflineBERT

简化版离线 Transformer 编码模型，模拟 BERT 特征提取逻辑，结构如下：

- 嵌入层（Embedding）：字符转稠密向量
- 多层 TransformerEncoder：3 层编码器、4 头注意力机制，提取文本上下文语义
- Dropout 层：抑制过拟合
- 全连接层：输出 3 维 Logits，对应 Positive（正向）、Negative（负向）、Neutral（中性）三类情感

输出经 Softmax 转为概率，最大概率值为文本分支置信度。

5.2 训练语料与编码规则

内置基础训练语料覆盖直播常见短句、网络用语：

- 正向：主播太厉害！、666、太强了、操作好丝滑、牛逼、主播加油等
- 负向：什么垃圾操作、太菜了吧、会不会玩、辣鸡、这都能输等
- 中性：还行吧、一般般、就这样、还行、中规中矩等

5.3 单模态文本缺陷分析

- 语义干扰：直播弹幕存在反讽、谐音、网络黑话，字面语义与真实情感相悖
- OCR 误差：画面模糊、文字倾斜、弹幕重叠导致识别错字、漏字，破坏语义
- 短句歧义：单字、极短弹幕语义模糊，分类置信度偏低

6 核心任务三：多模态融合策略（晚期动态加权融合）

6.1 融合方式选择：晚期融合

本系统采用特征级晚期融合：不融合网络中间特征，分别输出视觉七维概率向量、文本映射后七维概率向量，在决策层完成加权融合。

优势：两个模态模型独立训练、独立推理，解耦性强，单模态故障不导致整体系统崩溃，适配工程落地。

6.2 维度映射规则（代码固定映射矩阵）

文本模型输出为3维概率（正/负/中），视觉为7维概率（七类表情），首先完成维度对齐，定义映射矩阵TEXT_TO_7:

计算规则：文本七维概率 = 文本三维概率 × 映射矩阵，实现两模态维度统一。

6.3 动态权重逻辑（代码核心逻辑）

- 当文本OCR清晰、语义明确（文本置信度高），自动提升文本分支话语权
- 当人脸完整、表情清晰（视觉置信度高），自动提升视觉分支话语权
- 若某一分支失效（如无人脸/无文本），则另一分支结果直接作为最终结果
- 两分支同时有效时，按置信度自适应加权，平衡双方输出

6.4 融合后决策规则

融合得到七维概率向量后，取最大概率对应的表情标签作为系统最终情感输出，最大概率为整体融合置信度。

7 核心任务四：单模态与多模态性能对比分析

结合实验运行效果与理论分析，从准确率、鲁棒性、抗干扰能力、适用场景四个维度对比：

7.1 单模态视觉模型

- **优势：**主播面部清晰、光照良好时，表情识别直观、准确率高
- **劣势：**受光照、遮挡、姿态影响极大，复杂直播画面下误判率高
- **适用场景：**仅分析主播表情、无弹幕/字幕的纯视频场景

7.2 单模态文本模型

- **优势：**不受画面环境影响，可精准解读弹幕、字幕的文字语义
- **劣势：**无法感知主播真实状态，易被反讽、歧义语句误导，OCR 出错则完全失效
- **适用场景：**仅分析观众评论、无主播画面的纯文本场景

7.3 多模态融合模型

互补性体现：

- 画面质量差、人脸模糊时，自动加大文本权重，依靠文本语义决策
- 文本语义歧义、弹幕杂乱时，自动加大视觉权重，依靠主播表情决策
- 整体表现：平均准确率、场景鲁棒性全面优于任意单模态，可适配直播全场景
- 实验现象：正常场景下融合结果与单模态高置信度结果保持一致；干扰场景下，融合有效降低误判

7.4 量化现象总结

- **模态信息一致**（表情开心 + 弹幕正向）：三者输出结果完全统一，融合置信度高于单模态
- **单一模态低置信**（人脸模糊/文本歧义）：融合结果偏向另一高置信模态，纠正误判
- **双模态低置信**（画面差 + 文本乱）：融合结果偏向中性，避免极端误判

8 核心任务五：矛盾样本专项分析（表情与文本情感冲突）

8.1 典型矛盾场景

直播高频冲突场景：主播面带微笑（视觉识别为 **Happy** 开心，视觉置信度高），但弹幕大量负面评论（文本识别为 **Negative** 负向，文本置信度高）。

8.2 单模态表现

- **视觉单模态：**强制输出 **Happy**（开心），无法感知观众负面情绪，脱离直播整体语境
- **文本单模态：**强制输出对应负向映射表情，忽略主播真实神态，判断片面

8.3 多模态融合表现（结合代码动态权重）

- 两分支置信度均处于较高水平，权重均等参与计算
- 开心概率与负向对应表情概率相互抵消，融合后最大概率偏向 **Neutral**（中性），整体情感偏向中性偏负面
- 输出结果兼顾主播神态与观众评论，更贴合直播真实语境，避免单模态极端误判

8.4 多组冲突场景实验结论

- 视觉高置信、文本低置信：融合结果偏向主播表情
- 文本高置信、视觉低置信：融合结果偏向弹幕文本情感
- 双方置信度接近：结果收敛为中性，折中输出，符合客观场景

9 关键代码模块解析（结合报告功能对应关系）

本系统代码采用面向对象模块化编程，每个类对应报告中一个核心功能：

- `draw_chinese_text` 函数：解决 OpenCV 原生接口无法正常显示中文、出现乱码/问号的问题，基于 PIL 实现中文渲染，对应报告结果可视化模块。
- `EnhancedOfflineBERT+ EnhancedTokenizer`类：实现文本编码、语义提取、情感三分类，对应文本情感识别核心任务。
- `EmotionResNet`类：基于 ResNet18 改造的七分类表情模型，对应视觉情感识别核心任务。
- `YuNetFaceDetector`类：人脸检测主类，集成 YuNet 与 Haar 双检测器、帧缓存优化，对应人脸定位子任务。
- `MultiModalEmotionFusion`类：系统核心融合器，整合两大模型、维度映射、动态加权算法，对应多模态融合核心任务。
- `SyncOCR`类：同步 OCR、区域文本提取、文本清洗，对应字幕/弹幕文本提取模块。
- `process_video_stream` 函数：程序主处理流水线，串联所有模块、视频流循环、画面绘制，并以生成器形式逐帧输出标注结果。
- 前端界面（`app.py`）：基于 Gradio 搭建，负责文件上传、触发分析、实时显示视频流，实现人机交互。

10 前端交互界面设计与实现

10.1 技术选型与设计目标

前端采用 **Gradio** 框架，它是一个开源的 Python 库，能够快速构建机器学习模型的 Web 演示界面。选择 Gradio 的原因如下：

- 轻量级：无需编写 HTML/CSS/JavaScript，纯 Python 代码即可构建交互界面。
- 与后端无缝集成：可直接调用 Python 函数，支持生成器输出，天然适配视频流式返回。
- 跨平台：生成的 Web 页面可在任何现代浏览器中打开，支持局域网访问。
- 快速迭代：开发周期短，适合实验性系统的前端展示。

设计目标是为用户提供简洁直观的操作入口：

- 上传本地视频文件

- 一键启动实时情感分析
- 实时观看带标注结果的视频画面
- 无需任何命令行操作，降低使用门槛

10.2 界面布局与交互流程

界面组成：

- **标题区：**显示系统名称和简要说明。
- **左列（输入区）：**
 - `gr.Video`组件：支持拖拽或点击上传视频文件（常见格式如mp4、avi 等）。
 - `gr.Button`按钮：触发分析流程。
- **右列（输出区）：**
 - `gr.Image`组件：类型为numpy，以约 400 像素高度实时显示当前处理帧的标注结果。**交互**

流程：

1. 用户上传视频文件，组件返回视频文件路径。
2. 点击按钮，后端调用`process_video_stream(video_path)`函数。
3. 该函数是一个生成器（yield），逐帧返回处理后的 RGB 图像。
4. Gradio 自动轮询生成器，每当有新帧产出，即更新 `output_image`组件，形成流畅的“伪实时”播放效果。
5. 视频播放完毕或用户关闭页面，生成器自动终止，资源释放。

10.3 前后端数据交互机制

- 前端每次请求新帧时，生成器继续执行直至下一次 `yield`。
- `Gradio` 内部处理了生成器的迭代，确保不会阻塞主线程。
- 每帧处理耗时需控制在一定范围内（通常小于帧间隔时间），否则会出现卡顿。代码中通过跳帧（降低 OCR 和人脸检测频率）和缩放图像来优化速度。

10.4 可视化增强与用户体验

前端显示的画面中，后端已绘制了丰富的辅助信息，帮助用户直观理解识别结果：

- **人脸框**：绿色矩形框标注检测到的主播面部。
- **ROI 区域**：红色矩形框显示人脸检测区域；蓝色和黄色矩形框分别标示字幕和弹幕提取区域。
- **情感标签**：画面左上角显示融合后的最终情感（如“Happy (92.3%)”），并实时更新。
- **原始文本**：展示 OCR 识别出的字幕和弹幕内容（截断显示）。
- **单模态置信度**：可选显示视觉和文本分支的置信度，便于调试。

这些信息均由后端绘制后整体返回，前端仅负责展示，保证了界面简洁与渲染效率。

10.5 部署与访问方式

- **本地运行**：执行 `python app.py` 后，`Gradio` 会在本地启动一个 Web 服务，默认监听 `http://127.0.0.1:7860`。

11 实验结果与综合分析

11.1 整体实验结果

- **系统完整性**：完整实现视频读取、人脸检测、OCR 文本提取、单模态识别、多模态融合、可视化、前端交互全部功能，程序运行稳定、视频播放无明显卡顿
- **同情感场景**：文本与表情情感一致时，多模态识别结果稳定，置信度高于单模态
- **干扰场景**：光照、遮挡、文字歧义等干扰下，多模态融合抗干扰能力显著强于单模态
- **矛盾场景**：表情与文本情感冲突时，动态加权策略可实现折中决策，输出符合真实直播语境的结果

11.2 各模块运行效果

- **YuNet 人脸检测**：正常画面下人脸检出率高，降级 Haar 检测器后仍可基础使用
- **EasyOCR 离线识别**：清晰字幕、标准弹幕识别准确率高，小字、重叠文字识别效果下降
- **双模型推理**：CPU/GPU 均可正常推理，GPU 环境下帧率明显提升
- **动态融合算法**：权重自适应逻辑生效，无权重卡死、计算异常问题
- **前端交互**：Gradio 界面响应迅速，视频流实时性良好，用户操作直观便捷

12 系统现存问题与后续改进方向

12.1 当前存在的问题（结合代码与实验现象）

- **数据集局限**：文本、视觉模型训练数据集规模较小，直播小众网络用语、特殊表情覆盖不足，泛化能力有限
- **OCR 性能短板**：依赖画面清晰度，弹幕密集、文字倾斜、模糊画面下识别准确率大幅下降；未做文字纠偏处理
- **文本模型能力不足**：自研简化版 Transformer 结构简单，未使用大规模预训练 BERT 权重，复杂语义、反讽语句理解能力弱
- **模态融合层次单一**：仅采用决策层晚期融合，未利用特征层融合，模态深层关联特征未被挖掘
- **模态种类缺失**：当前仅视觉、文本双模态，未利用直播音频（主播语气、观众声音）信息
- **人脸策略**：仅固定全屏 ROI 检测人脸，若主播位置偏移则可能漏检
- **前端功能有限**：当前仅支持视频上传和播放，缺乏进度控制、暂停/继续、结果导出（如 CSV）等高级交互功能

12.2 优化与改进方案

- **数据与模型升级**: 扩充直播弹幕、表情数据集, 增加反讽、谐音、方言类文本样本; 替换离线简易 Transformer, 引入正式预训练 BERT 模型, 提升文本语义理解能力
- **OCR 模块优化**: 增加图像二值化、去噪、倾斜校正等预处理, 接入文字纠错模型, 提升复杂画面文本识别率
- **融合算法升级**: 引入注意力机制, 自动学习不同场景下模态重要性, 替代简单线性加权
- **模态拓展**: 新增语音情感识别模块, 构建视觉+文本+语音三模态融合系统, 全方位解析直播情感
- **人脸检测优化**: 取消固定 ROI 限制, 增加多人脸筛选、主播身份区分逻辑, 适配主播移动、多人同框场景
- **前端增强**: 添加播放控制条、帧率显示、结果统计图表、一键导出分析报告等功能, 提升用户体验

13 实验总结

本实验基于 Python 与深度学习技术, 结合 YuNet、ResNet18、简易 Transformer、EasyOCR 等工具, 成功搭建一套面向直播场景的双模态情感识别系统, 完整完成题目要求的五大核心任务, 并额外构建了基于 Gradio 的前端交互界面。

系统通过视觉表情识别捕捉主播外在情绪, 通过文本情感识别解读观众弹幕与字幕语义, 再依托动态加权晚期融合策略实现双模态信息互补。前端界面提供视频上传、一键启动、实时画面显示等便捷操作, 使非技术用户也能轻松使用系统。实验证明: 相较于单一视觉或文本模型, 多模态融合方案在准确率、鲁棒性、复杂场景适应性上具备明显优势, 尤其在表情与文本情感矛盾的典型直播场景中, 能够做出更贴合实际语境的判断。

本次实验不仅完成了算法层面的验证, 还落地为工程化可运行代码, 实现了视频实时推理、可视化交互等实用功能, 为直播情感分析、人机情感交互、直播内容风控等后续研究与工程应用提供了可行的实践方案。同时, 本实验也梳理出模型、数据、算法以及前端交互上的不足, 为后续多模态情感识别系统的迭代优化指明了方向。

参考文献

- [1] OpenCV. Face Detection YuNet model. https://github.com/opencv/opencv_zoo
- [2] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]. Proceedings of the IEEE conference on computer vision and pattern recognition, 2016: 770-778.
- [3] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]. Advances in neural information processing systems, 2017: 5998-6008.
- [4] Jaided AI. EasyOCR: Ready-to-use OCR with 80+ supported languages. <https://github.com/JaidedAI/EasyOCR>
- [5] Gradio. Build and share delightful machine learning apps. <https://gradio.app>